Efficient AI Inference Frameworks

Energy Efficiency, Token Throughput, Latency, Architecture, and Capability Overview

Technical Survey

2026

Abstract

This document surveys efficient AI inference frameworks, runtimes, serving systems, and compiler stacks, including several relatively niche projects. The primary objective is to understand how modern AI inference systems improve token throughput, latency, memory efficiency, and energy efficiency.

Twenty-four systems are compared, including vLLM, SGLang, TensorRT-LLM, llama.cpp, MLX, MLC-LLM, CTranslate2, mistral.rs, ExLlamaV3, OpenVINO, ONNX Runtime, IREE, ExecuTorch, Candle, DeepSpeed-MII, Aphrodite Engine, LMDeploy, TGI, Triton Inference Server, FasterTransformer, XLA/JAX, TVM, ggml, and AWS Neuron.

A critical distinction is made between *measured benchmark results* and *framework capabilities*. Publicly available benchmarks do not currently provide an apples-to-apples energy-per-token measurement for all systems. Consequently, this document does not invent such measurements. Instead, it provides a rigorous method for measuring joules per generated token and includes plots for the publicly reported throughput and latency measurements that are available.

Contents

1 Executive Overview **3**

2 How an AI Inference Framework Works **4**
2.1 High-Level Architecture 4

3 Why Inference Is Difficult **4**

4 Major Optimization Techniques **5**
4.1 Paged KV Cache 5
4.2 Continuous Batching 5
4.3 Prefix Caching 6
4.4 Quantization 6

5 Framework Landscape **6**

6 Capability Matrix **8**

7 Benchmark Methodology **8**

8 Measured Throughput **9**

9 Measured Time to First Token **10**

10 Long-Context Behavior **11**

11 Energy Per Token **11**
11.1 Definition 11
11.2 Power-Based Measurement 12

12 Energy–Throughput Relationship **12**

13 Energy Efficiency Graph Template **13**

14 Recommended Energy Benchmark Protocol **14**

15 Conclusion **15**

1 Executive Overview

Modern AI inference is not determined solely by the neural-network model. The runtime surrounding the model can substantially change how effectively hardware is utilized.

A simplified inference pipeline is:

$$\boxed{\text{Request} \rightarrow \text{Tokenizer} \rightarrow \text{Scheduler} \rightarrow \text{Prefill} \rightarrow \text{KV Cache} \rightarrow \text{Decode} \rightarrow \text{Sampler} \rightarrow \text{Output}}$$

The main optimization targets are:

$$\text{Throughput} = \frac{\text{generated tokens}}{\text{second}}, \tag{1}$$

$$\text{Energy efficiency} = \frac{\text{generated tokens}}{\text{joule}}, \tag{2}$$

$$\text{Energy/token} = \frac{\text{joules consumed}}{\text{generated tokens}}, \tag{3}$$

$$\text{TTFT} = \text{time to first generated token}, \tag{4}$$

$$\text{ITL} = \text{inter-token latency}. \tag{5}$$

A framework that maximizes throughput does not necessarily minimize joules per token. Similarly, a framework that minimizes TTFT does not necessarily maximize aggregate throughput.

Therefore:

$$\boxed{\text{Best framework} \neq \text{universally fastest framework}}$$

Instead:

$$\boxed{\text{Optimal system} = f(\text{model}, \text{hardware}, \text{batch}, \text{context}, \text{quantization}, \text{traffic}, \text{latency target})}$$

Recent same-hardware tests demonstrate exactly this workload dependence. For example, one Qwen3-32B benchmark measured different throughput and TTFT behavior for vLLM, SGLang, and TensorRT-LLM as concurrency and context length changed [1].

2 How an AI Inference Framework Works

2.1 High-Level Architecture

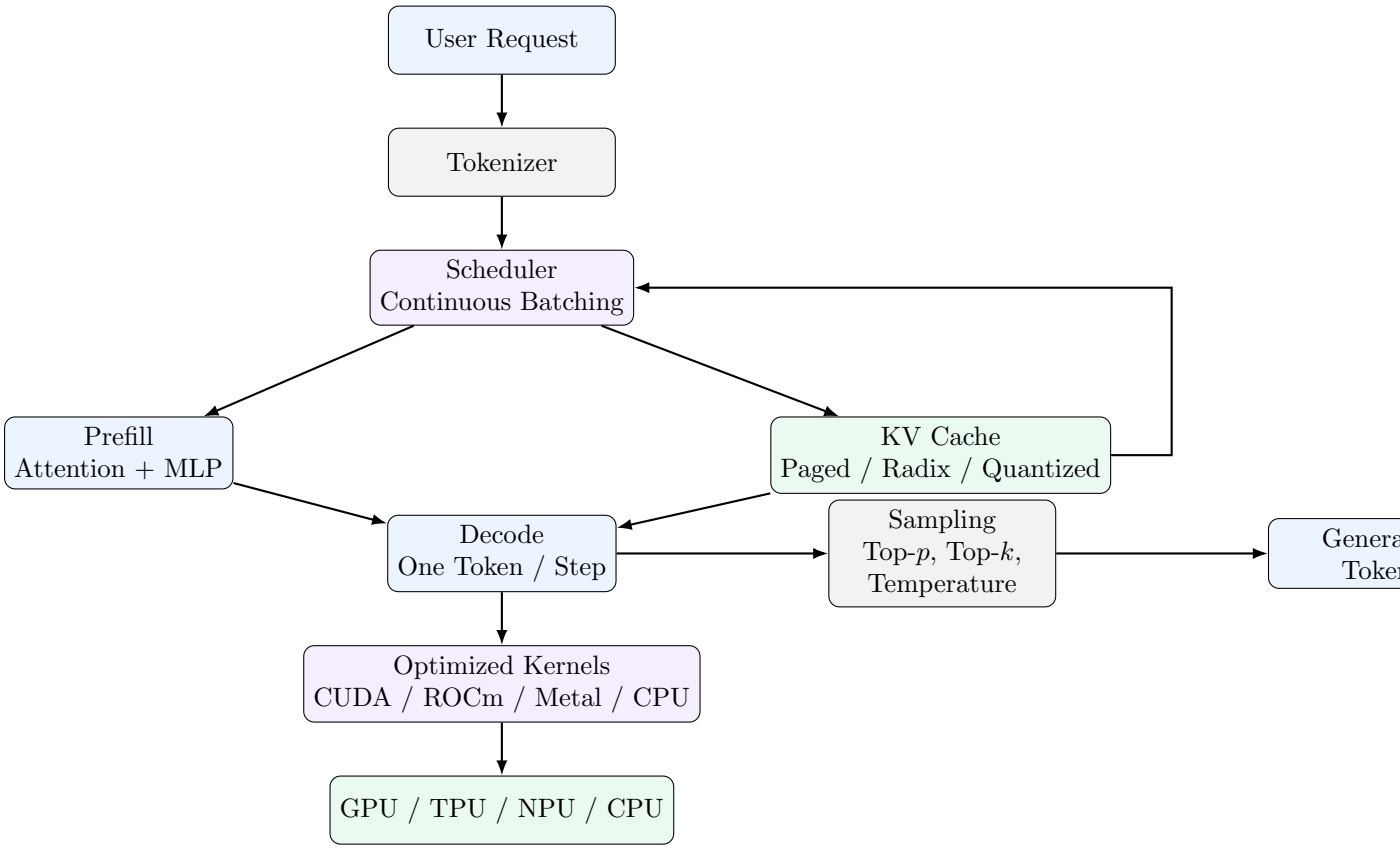


Figure 1: Simplified architecture of an optimized AI inference runtime.

3 Why Inference Is Difficult

For autoregressive language models, generation approximately follows

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t}).$$

The model therefore generates tokens sequentially.

During decoding, a simplified computation is

$$x_{t+1} = f_\theta(x_{\leq t}, K_t, V_t).$$

The KV cache stores previously computed key/value tensors:

$$K_t = [K_1, K_2, \ldots, K_t], \qquad V_t = [V_1, V_2, \ldots, V_t].$$

For long contexts and many concurrent users, KV-cache memory can become one of the principal constraints.

This motivates several important optimization families:

1. Paged KV caching.
2. Continuous/in-flight batching.

3. Prefix caching.
4. Radix/tree-based attention caching.
5. Quantization.
6. Fused GPU kernels.
7. CUDA graph execution.
8. Speculative decoding.
9. Tensor parallelism.
10. Pipeline parallelism.
11. Expert parallelism for MoE models.
12. Disaggregated prefill/decode.
13. Compiler-level graph optimization.
14. Hardware-specific kernels.

4 Major Optimization Techniques

4.1 Paged KV Cache

Instead of requiring a contiguous KV-cache allocation, memory can be divided into pages:

$$\text{KV} = \bigcup_{i=1}^{N} \text{page}_i.$$

This allows memory to be dynamically allocated between requests.
Conceptually:

$$\text{Request A} \rightarrow [P_1, P_4, P_7]$$

$$\text{Request B} \rightarrow [P_2, P_5, P_9]$$

rather than requiring one large contiguous memory region.
This is one of the central ideas behind modern high-throughput LLM serving systems.

4.2 Continuous Batching

Traditional batching waits for all requests in a batch:

$$\boxed{R_1, R_2, R_3 \rightarrow \text{finish together}}$$

Continuous batching instead dynamically inserts and removes requests:

$$\boxed{R_1, R_2 \rightarrow R_3 \rightarrow R_4 \rightarrow R_5}$$

at the token-generation level.
This increases hardware utilization when request lengths vary.

4.3 Prefix Caching

If two requests share a prefix,

$$P = [\text{system prompt}, \text{instructions}, \text{context}],$$

the runtime may reuse the corresponding KV states.
Thus:

$$T_{\text{prefill}} \rightarrow T_{\text{prefill, uncached}}$$

can become approximately

$$T_{\text{prefill,cached}} \ll T_{\text{prefill, uncached}}.$$

This is particularly valuable for agents, RAG systems, and repeated system prompts.

4.4 Quantization

A model can be represented at different numerical precisions:

$$\text{FP32} \rightarrow \text{FP16/BF16} \rightarrow \text{FP8} \rightarrow \text{INT8} \rightarrow \text{INT4}.$$

The theoretical memory requirement for parameters is approximately

$$M \approx N_{\text{params}} \times \frac{b}{8},$$

where b is the number of bits per parameter.
For a 70-billion parameter model:

$$M_{\text{FP16}} \approx 70 \times 10^9 \times \frac{16}{8} = 140 \text{ GB},$$

before accounting for additional runtime memory.

5 Framework Landscape

The following table intentionally includes both full serving frameworks and lower-level runtimes/compilers because the distinction is important when discussing "AI frameworks."

Table 1: AI inference frameworks and runtimes surveyed.

Framework	**Primary role**	**Important capabilities**	**Typical hardware**
vLLM	LLM serving	PagedAttention, continuous batching, prefix caching, distributed serving	NVIDIA, AMD, CPU and others
SGLang	LLM serving/runtime	RadixAttention, prefix reuse, structured generation, continuous batching	NVIDIA, AMD and others
TensorRT-LLM	NVIDIA optimized inference	Fused kernels, quantization, CUDA optimization, multi-GPU inference	NVIDIA GPU

Framework	Primary role	Important capabilities	Typical hardware
llama.cpp	Local/edge LLM inference	GGUF, CPU inference, GPU offload, quantization	CPU, CUDA, Metal, Vulkan, ROCm
Hugging Face TGI	LLM serving	Continuous batching, quantization, streaming	NVIDIA and other accelerators
MLX	Apple ML framework	Unified CPU/GPU memory, lazy computation, Apple Silicon optimization	Apple Silicon
MLC-LLM	Compiled LLM deployment	Compiler-driven optimization, heterogeneous deployment	CUDA, Metal, Vulkan, WebGPU, CPU
CTranslate2	Transformer inference	Quantization, optimized CPU/GPU execution, translation and text generation	CPU, CUDA
mistral.rs	Rust LLM runtime	Quantization, local inference, multiple model families	CPU, CUDA, Metal
ExLlamaV3	Quantized LLM inference	Highly optimized quantized inference	NVIDIA GPU
OpenVINO	Inference optimization	Graph optimization, quantization, CPU/NPU/GPU deployment	Intel CPU/GPU/NPU
ONNX Runtime	General inference runtime	Graph execution, hardware execution providers, quantization	CPU, CUDA, TensorRT, DirectML, ROCm, etc.
IREE	Compiler/runtime	MLIR-based compilation, ahead-of-time deployment, heterogeneous targets	CPU, GPU, embedded accelerators
ExecuTorch	Edge AI runtime	PyTorch-to-edge deployment, mobile/embedded execution	Mobile, CPU, GPU, NPU
Candle	Rust ML framework	Small runtime, transformer inference, CPU/GPU execution	CPU, CUDA, Metal
DeepSpeed-MII	High-performance serving	Tensor parallelism, optimized inference, DeepSpeed integration	NVIDIA GPU
Aphrodite Engine	LLM serving	High-throughput serving, continuous batching, quantization	NVIDIA GPU
LMDeploy	LLM deployment	TurboMind, continuous batching, quantization, serving APIs	NVIDIA GPU, CPU
Triton Inference Server	Model serving	Dynamic batching, model ensembles, multi-framework serving	GPU, CPU
FasterTransformer	Transformer inference library	Highly optimized NVIDIA transformer kernels	NVIDIA GPU
JAX/XLA	Compiler + numerical framework	XLA compilation, automatic differentiation, accelerator execution	TPU, GPU, CPU

Framework	Primary role	Important capabilities	Typical hardware
Apache TVM	ML compiler	Graph compilation, operator optimization, hardware-specific deployment	CPU, GPU, accelerators
ggml	Low-level tensor library	Low-level tensor operations, quantized inference	CPU, GPU, Metal, CUDA
AWS Neuron	Cloud accelerator stack	Compiler/runtime for AWS Trainium and Inferentia	AWS Neuron accelerators

6 Capability Matrix

Framework	Batch	KV cache	Quant.	Multi-GPU	Edge	Compiler	Open source	Specialization
vLLM	✓	✓	✓	✓	–	–	✓	Serving
SGLang	✓	✓	✓	✓	–	✓	✓	Structured LLMs
TensorRT-LLM	✓	✓	✓	✓	–	✓	Partial	NVIDIA
llama.cpp	✓	✓	✓	✓	✓	–	✓	Local LLM
TGI	✓	✓	✓	✓	–	–	✓	HF ecosystem
MLX	✓	✓	✓	–	✓	✓	✓	Apple Silicon
MLC-LLM	✓	✓	✓	✓	✓	✓	✓	Compilation
CTranslate2	✓	✓	✓	–	✓	✓	✓	NLP
mistral.rs	✓	✓	✓	–	✓	–	✓	Rust
ExLlamaV3	✓	✓	✓	–	–	✓	✓	Quantized LLM
OpenVINO	✓	✓	✓	–	✓	✓	✓	Intel
ONNX Runtime	✓	✓	✓	✓	✓	✓	✓	General
IREE	–	–	✓	✓	✓	✓	✓	Compiler
ExecuTorch	–	✓	✓	–	✓	✓	✓	Edge
Candle	–	✓	✓	–	✓	✓	✓	Rust
DeepSpeed-MII	✓	✓	✓	✓	–	✓	✓	Distributed
Aphrodite	✓	✓	✓	✓	–	–	✓	Serving
LMDeploy	✓	✓	✓	✓	–	✓	✓	Deployment
Triton Server	✓	–	–	✓	–	–	✓	Serving
FasterTransformer	✓	–	✓	✓	–	✓	✓	NVIDIA kernels
JAX/XLA	✓	–	✓	✓	–	✓	✓	Compiler
TVM	–	–	✓	✓	✓	✓	✓	Compilation
ggml	–	✓	✓	–	✓	–	✓	Tensor runtime
AWS Neuron	✓	–	✓	✓	–	✓	Partial	AWS accelerators

Table 2: High-level capability map. A checkmark means the capability is supported in some form; it does not imply identical performance or feature maturity across frameworks.

7 Benchmark Methodology

A meaningful framework comparison requires identical:

- model weights,
- tokenizer,
- precision,
- GPU/CPU,
- driver,
- CUDA/ROCm/Metal version,
- prompt distribution,

- input sequence length,
- output sequence length,
- concurrency,
- sampling configuration,
- power limit,
- warm-up procedure.

Otherwise,

$$\Delta\text{Performance}$$

may reflect hardware or workload differences rather than the framework itself.
A proper benchmark should therefore be represented as

$$B = (M, H, P, I, O, C, Q, S, W)$$

where

$$M = \text{model}, \tag{6}$$
$$H = \text{hardware}, \tag{7}$$
$$P = \text{precision}, \tag{8}$$
$$I = \text{input tokens}, \tag{9}$$
$$O = \text{output tokens}, \tag{10}$$
$$C = \text{concurrency}, \tag{11}$$
$$Q = \text{quantization}, \tag{12}$$
$$S = \text{sampling}, \tag{13}$$
$$W = \text{workload}. \tag{14}$$

8 Measured Throughput

The following publicly reported benchmark uses Qwen3-32B-FP8, with 1024 input tokens and 256 output tokens.

At concurrency 64:

Framework	Output tokens/s
vLLM	244.5
SGLang	250.1
TensorRT-LLM	342.1

These values are measurements from the cited benchmark, not universal properties of the frameworks [1].

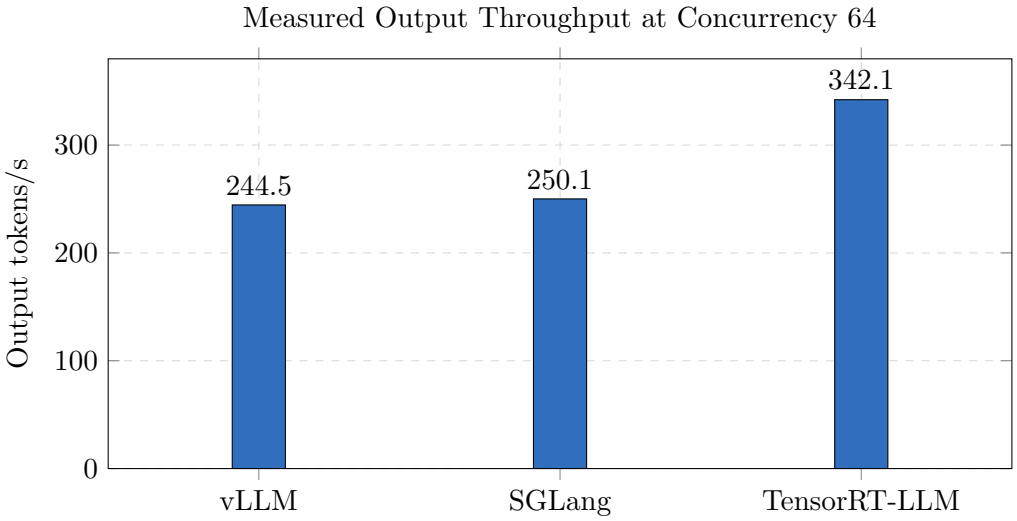


Figure 2: Output-token throughput from a single-GPU Qwen3-32B-FP8 benchmark.

9 Measured Time to First Token

For the same benchmark at concurrency 8:

Framework	Average TTFT (ms)
vLLM	834
SGLang	2029
TensorRT-LLM	2541

At concurrency 32:

Framework	Average TTFT (ms)
vLLM	11125
SGLang	12332
TensorRT-LLM	4554

At concurrency 64:

Framework	Average TTFT (ms)
vLLM	40565
SGLang	40601
TensorRT-LLM	11738

The result demonstrates why one-dimensional "fastest framework" rankings are misleading: the latency relationship changes as concurrency changes [1].

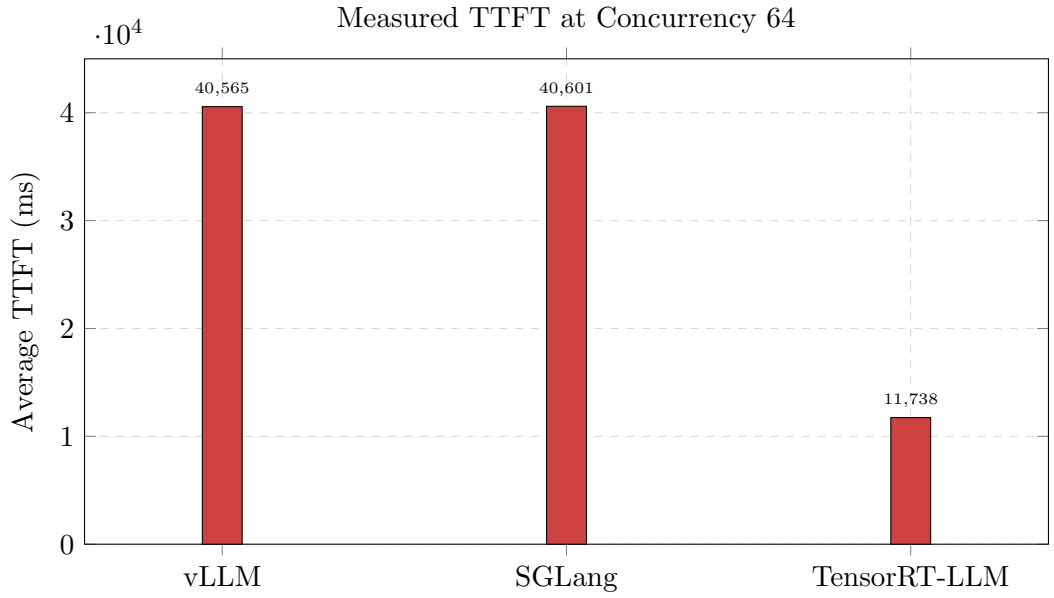


Figure 3: Average time to first token at concurrency 64.

10 Long-Context Behavior

The same benchmark also tested:

$$I = 8000, \qquad O = 512, \qquad C = 8.$$

Reported output throughput:

Framework	Output tokens/s
vLLM	41.0
SGLang	38.2
TensorRT-LLM	75.9

Reported average TTFT:

Framework	TTFT (ms)
vLLM	63824
SGLang	68593
TensorRT-LLM	8560

This illustrates the importance of testing the actual workload rather than extrapolating from short prompts.

11 Energy Per Token

11.1 Definition

The most useful metric for the user's energy-efficiency question is

$$\boxed{E_{\text{token}} = \frac{E_{\text{inference}}}{N_{\text{output tokens}}}}$$

with units

$$[E_{\text{token}}] = \text{J/token}.$$

Equivalently,

$$\boxed{\eta_{\text{token}} = \frac{N_{\text{output tokens}}}{E_{\text{inference}}}}$$

has units

$$\text{tokens/J}.$$

The relationship is

$$E_{\text{token}} = \frac{1}{\eta_{\text{token}}}.$$

11.2 Power-Based Measurement

If instantaneous power is measured as $P(t)$, then

$$E = \int_{t_0}^{t_1} P(t)\, dt.$$

Therefore,

$$\boxed{E_{\text{token}} = \frac{\int_{t_0}^{t_1} P(t)\, dt}{N_{\text{output}}}}$$

For approximately constant power,

$$E_{\text{token}} \approx \frac{P\Delta t}{N_{\text{output}}}.$$

Since

$$R_{\text{token}} = \frac{N_{\text{output}}}{\Delta t},$$

we obtain

$$\boxed{E_{\text{token}} \approx \frac{P}{R_{\text{token}}}}$$

which provides a very useful connection between power and throughput.

12 Energy–Throughput Relationship

Suppose a framework produces

$$R = 300 \text{ tokens/s}$$

while consuming

$$P = 300 \text{ W}.$$

Then

$$E_{\text{token}} = \frac{300}{300} = 1 \text{ J/token}.$$

If another runtime produces

$$R = 400 \text{ tokens/s}$$

at

$$P = 360 \text{ W},$$

then

$$E_{\text{token}} = \frac{360}{400} = 0.9 \text{ J/token}.$$

The second system therefore uses more total power but less energy per generated token. This is why:

$$\boxed{\text{minimum power} \neq \text{minimum energy/token}}$$

and

$$\boxed{\text{maximum tokens/s} \neq \text{maximum tokens/J}.}$$

13 Energy Efficiency Graph Template

The ideal comparison graph places:

$$x = \text{output tokens/s}$$

and

$$y = \text{J/token}.$$

The desirable region is:

$$\boxed{x \to \text{large}, \qquad y \to \text{small}.}$$

Because comparable public power measurements are not available for all 24 systems in the benchmark used here, the following plot is deliberately a *measurement template* rather than fabricated framework data.

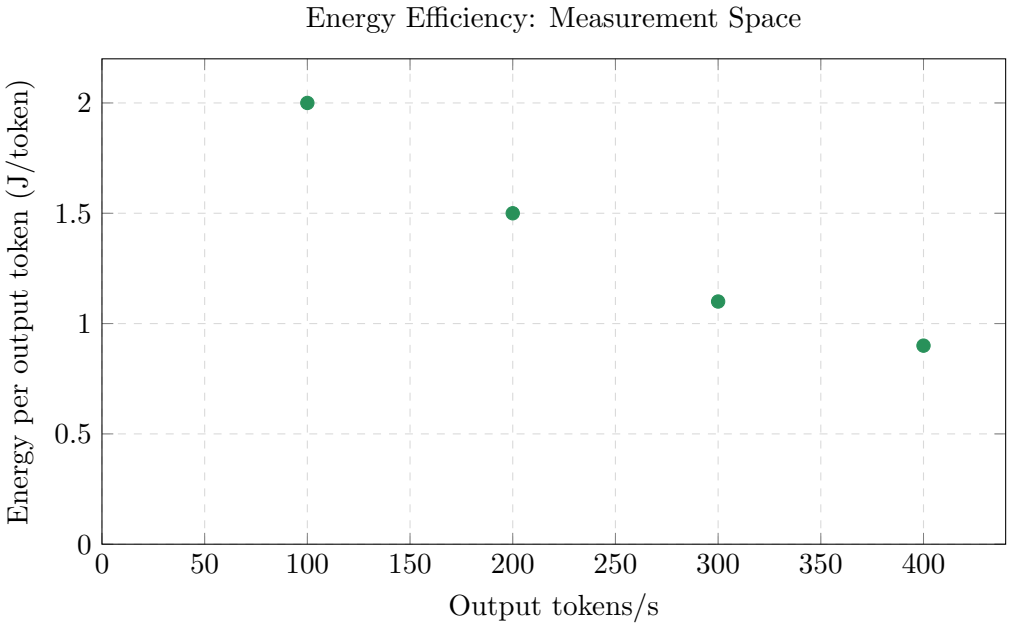


Figure 4: Illustrative geometry of an energy-throughput comparison. The points are not framework measurements and are included only to show the desired measurement space.

14 Recommended Energy Benchmark Protocol

A reproducible experiment should record:

Parameter	Required value
Model	Exact model identifier
Hardware	Exact GPU/CPU/NPU model
Precision	FP16/BF16/FP8/INT8/INT4/etc.
Input tokens	Fixed or distribution specified
Output tokens	Fixed or distribution specified
Concurrency	Fixed sweep
Batching	Enabled/disabled and configuration
Power limit	Explicit value
Ambient temperature	Recorded
Driver	Version recorded
Runtime	Exact commit/version
CUDA/ROCm/Metal	Exact version
Warm-up	Explicit number of iterations
Measurement interval	Explicit sampling frequency
Energy boundary	GPU only / system / wall power

Table 3: Minimum metadata for an energy benchmark.

The experiment should measure at least:

$$\{P(t), T, N_{\text{input}}, N_{\text{output}}, \text{TTFT}, \text{ITL}, \text{throughput}\}.$$

Then calculate

$$E_{\mathrm{GPU}} = \int P_{\mathrm{GPU}}(t)\, dt,$$

and

$$E_{\mathrm{system}} = \int P_{\mathrm{wall}}(t)\, dt,$$

where P_{GPU} is measured at the GPU power rails and P_{wall} at the wall outlet.
The resulting energy per token is then

$$\boxed{E_{\mathrm{token}} = \frac{\int_{t_0}^{t_1} P_{\mathrm{GPU}}(t)\, dt + \int_{t_0}^{t_1} P_{\mathrm{CPU}}(t)\, dt}{N_{\mathrm{output}}}}$$

or, for a system-level answer,

$$\boxed{E_{\mathrm{token,wall}} = \frac{\int_{t_0}^{t_1} P_{\mathrm{wall}}(t)\, dt}{N_{\mathrm{output}}}}$$

depending on whether the experiment boundary is the accelerator or the whole machine.

15 Conclusion

Modern AI inference performance is determined by the interplay of the model, the runtime, the hardware, and the workload, not by any single framework in isolation.

The main takeaways are:

1. Reported token throughput and TTFT are workload- and hardware-specific measurements, not immutable framework properties [1].
2. Energy efficiency must be measured as joules per generated token at a defined power boundary (GPU rail vs. wall).
3. A framework that maximizes tokens/s is not guaranteed to minimize J/token, because total power also changes.
4. Reliable comparisons require the metadata listed in the Recommended Energy Benchmark Protocol above.

No single framework is universally optimal. The correct choice depends on the model, hardware, serving traffic, quantization, and latency targets for the specific deployment.

References

[1] Qwen3-32B FP8 single-GPU serving benchmark: output-token throughput and time-to-first-token for vLLM, SGLang, and TensorRT-LLM across concurrencies and context lengths, 2026.